Ezairo® 7150 SL Remote Microphone Reference Design Firmware Manual

M-20825-001
December 2015

© SCILLC, 2015
Previous Edition © 2015
"All Rights Reserved"

ON Semiconductor®

Ezairo 7150 SL Remote Microphone Reference Design Firmware Manual

ON Semiconductor and ON are registered trademarks of Semiconductor Components Industries, LLC (SCILLC). Ezairo is a registered trademark of SCILLC. Cortex and ARM are registered trademarks of ARM Limited. Bluetooth is a registered trademark of Bluetooth SIG, Inc. SCILLC owns the rights to a number of patents, trademarks, copyrights, trade secrets, and other intellectual property. A listing of SCILLC's product/patent coverage may be accessed at www.onsemi.com/site/pdf/Patent-Marking.pdf. SCILLC reserves the right to make changes without further notice to any products herein. SCILLC makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does SCILLC assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation special, consequential or incidental damages. "Typical" parameters which may be provided in SCILLC data sheets and/or specifications can and do vary in different applications and actual performance may vary over time. All operating parameters, including "Typicals" must be validated for each customer application by customer's technical experts. SCILLC does not convey any license under its patent rights nor the rights of others. SCILLC products are not designed, intended, or authorized for use as components in systems intended for surgical implant into the body, or other applications intended to support or sustain life, or for any other application in which the failure of the SCILLC product could create a situation where personal injury or death may occur. Should Buyer purchase or use SCILLC products for any such unintended or unauthorized application, Buyer shall indemnify and hold SCILLC and its officers, employees, subsidiaries, affiliates, and distributors harmless against all claims, costs, damages, and expenses, and reasonable attorney fees arising out of, directly or indirectly, any claim of personal injury or death associated with such unintended or unauthorized use, even if such claim alleges that SCILLC was negligent regarding the design or manufacture of the part. SCILLC is an Equal Opportunity/ Affirmative Action Employer. This literature is subject to all applicable copyright laws and is not for resale in any manner.

PUBLICATION ORDERING INFORMATION

LITERATURE FULFILLMENT:
Literature Distribution Center for ON Semiconductor
P.O. Box 5163, Denver, Colorado 80217 USA
Phone: 303-675-2175 or 800-344-3860 Toll Free USA/Canada
Fax: 303-675-2176 or 800-344-3867 Toll Free USA/Canada
Email: orderlit@onsemi.com

N. American Technical Support: 800-282-9855 Toll Free USA/Canada
Europe, Middle East and Africa Technical Support: Phone: 421 33 790 2910
Japan Customer Focus Center: Phone: 81-3-5817-1050

ON Semiconductor Website: www.onsemi.com

Order Literature: http://www.onsemi.com/orderlit

For additional information, please contact your local Sales Representative

Table of Contents

1. Introduction 5
1.1 Purpose 5
1.2 Intended Audience 5
1.3 Conventions 5
1.4 Manual Organization 5
1.5 Further Reading 6
2. System Level Overview 7
3. CFX Processor 10
3.1 Powering the nRF51822 Radio IC 10
3.2 Switching Between Audio Sources 10
3.3 Toggling Audio Stream 10
3.4 GPIO Monitoring 10
4. ARM Cortex-M3 Processor 11
4.1 System Initialization 11
4.1.1 Hardware Initialization 11
4.1.2 Radio Initialization 11
4.2 main loop 12
4.3 Button Flags 12
4.4 I2C Flags 12
4.5 I2C Queue Management 13
4.6 Ezairo 7150 SL 12C Library 13
4.6.1 Overview 13
4.6.2 Receiving a Command 14
4.6.3 Sending a Command 15
4.6.4 I2C Protocol 15
4.6.5 I2C Library Functions 19
4.6.5.1 I2C Initialize 20
4.6.5.2 I2C IRQHandler 20
4.6.5.3 I2C Clear Interrupt 20
4.6.5.4 I2C Set Interrupt 21
4.6.5.5 I2C Send Bytes 21
4.6.5.6 I2C Queue Put 21
4.6.5.7 I2C Queue Get 22
4.6.5.8 I2C Queue Status 22
4.6.5.9 I2C Queue Clear 23
4.6.5.10 I2C Set Flag 23
4.6.5.11 I2C Get Flag 23
4.6.5.12 I2C Hard Reset 24
4.6.5.13 I2C Tx Reset 24
4.6.5.14 I2C Rx Reset 24
4.6.6 Ezairo 7150 SL Library 25
4.6.6.1 Overview 25
4.7 BLE Library Functions 25
4.7.1 Configure Audio Stream 25

4.7.2 Change Radio Name 26
4.7.3 Initialize Radio 26
4.7.4 BLE On 26
4.7.5 BLE Off 27
4.7.6 Stream On 27
4.7.7 Stream Off 27
4.7.8 nRF51822 Sleep 28
5. NRF51822 Radio IC 29

CHAPTER 1
Introduction

1.1 PURPOSE

This document provides information for the Ezairo® 7150 SL remote microphone sample firmware. The firmware samples are divided into the following main modules:

- CFX Processor
  - Powering the nRF51822 radio IC
  - Audio Source
  - General Purpose Input/Output (GPIO) Monitoring- Phase-Locked Loop (PLL) Synchronization

- ARM® Cortex®-M3 Processor
  - System Initialization
  - Main System Loop
  - I^2C Slave Module
  - Bluetooth® Low Energy (BLE) Module
  - Serial Peripheral Interface (SPI) Module

This document discusses the operation of the above sections and covers how to modify the firmware to suit the intended application. The base application as configured implements the following:

- Radio power-up
- Application and peripheral initializations
- Radio initialization
- Microphone and 3.5 mm plug audio source switching
- Wireless audio streaming
- User feedback through LEDs
- User input through a push-button

1.2 INTENDED AUDIENCE

This manual is for engineers, developers, and anyone else who requires technical information for the firmware.The reader is expected to be familiar with the Ezairo 7100 Evaluation and Development Kit (EDK).

1.3 CONVENTIONS

The following conventions are used in this manual to signify particular types of information:

- File and path names, or any portion or them, are displayed in *italics*.
- Code samples and code snippets are displayed in a `monospace font`.

1.4 MANUAL ORGANIZATION

The *Ezairo 7150 SL Remote Microphone Reference Design Firmware Manual* contains the following chapters:

- **Chapter 1: Introduction**, describes the purpose and organization of this manual, the intended audience, and provides a list of recommended reading for more information.
- **Chapter 2: System Level Overview**, provides an overview of the Ezairo 7150 SL system.
- **Chapter 3: CFX Processor**, describes the capabilities of the CFX processor. It also provides reference information about the functions that are available to make programming the processor easier.

- **Chapter 4: ARM Cortex-M3 Processor**, describes the capabilities of the ARM Cortex-M3 processor. It also provides reference information about the functions that are available to make programming the processor easier.
- **Chapter 5: NRF51822 Radio IC**, describes the features that are available through this System-on-Chip (SoC).

1.5 Further Reading

For more information, refer to the following documents:

- *Communication Accelerator Adaptor Manual*, which is installed with the Communication Accelerator Adaptor (CAA) software
- *Integrated Development Environment User's Guide for Ezairo 7100*
- *Introduction to Ezairo 7100 Programming*
- *CFX DSP Architecture Manual*
- *Hardware Reference Manual for Ezairo 7100*
- *HEAR Configurable Accelerator Reference Manual*
- *Firmware Reference Manual for Ezairo 7100*
- *SK4 ELF Toolchain Reference for Ezairo 7100*
- *Ezairo 7150 SL Datasheet*
- *EA2M Datasheet* (Available at www.onsemi.com)
- *Ezairo 7150 SL BTE Hearing Aid Reference Design User Manual*
- *nRF51822 Product Specification Version 3.1*

CHAPTER 2

System Level Overview

The complete system consists of the Ezairo 7150 SL enabled hearing aid, the BLE control device (e.g., BLE-enabled cell phone), and the remote microphone.The remote microphone must be used in conjunction with an Ezairo 7150 SL-based hearing aid to achieve audio streaming. The remote microphone samples the incoming audio, either from microphones or analog input, and broadcasts this audio stream to the hearing aid devices within range. To locate the audio stream, the hearing aid must be configured to receive using the same parameters as the transmitter on the remote microphone.

The Ezairo 7150 SL hybrid in the remote microphone consists of the Ezairo 7100 digital signal processor and the nRF51822 radio IC. Ezairo 7100 and the radio use an I^2C profile to send commands back and forth, and an SPI interface to transfer the audio samples from the DSP to the radio.

Finally, the Ezairo 7100 DSP makes use of two of its processors: the CFX (audio DSP), and an ARM Cortex-M3 processor (control) to implement the firmware. The CFX is responsible for sampling and encoding the audio, while the ARM Cortex-M3 processor handles user interaction and communication with the nRF51822 radio IC using the I^2C and SPI interfaces.The complete system overview of the remote microphone is shown in Figure 1.

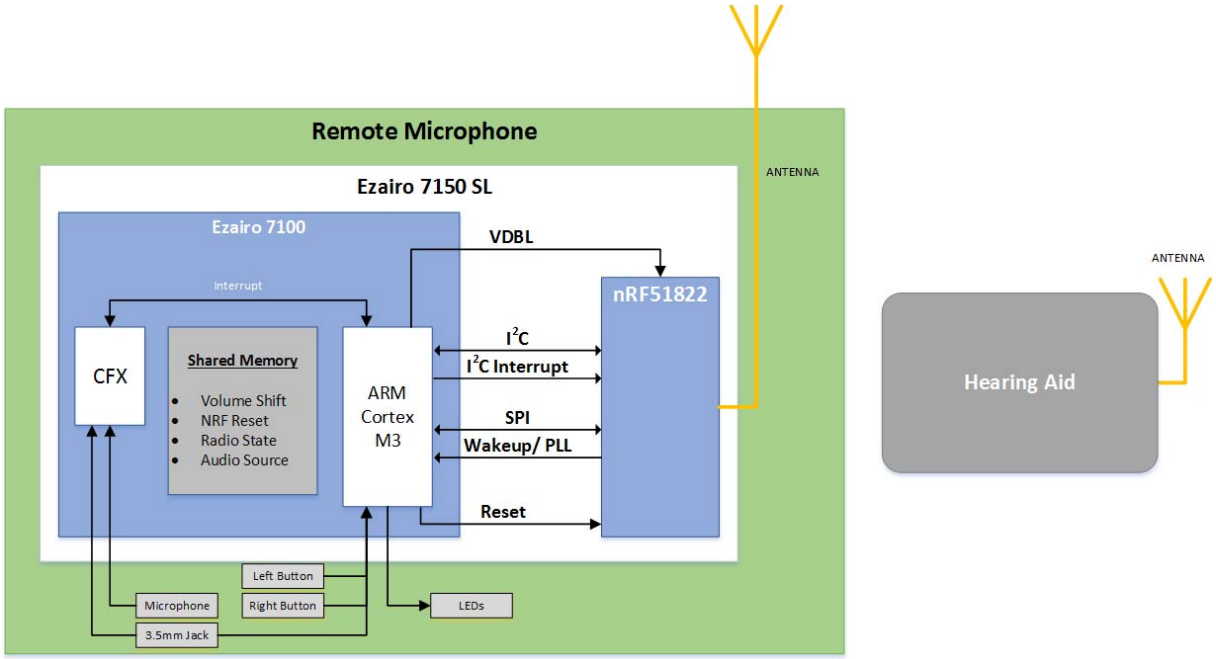


Figure 1. Remote Microphone System Overview

The processors and radio IC within the remote microphone interact as follows:

- A push-button is used for direct user input to the ARM Cortex-M3 processor. The button presses are debounced, and presses of varying duration can be detected.
- The CFX can sample audio from either the on-board microphones or from a 3.5 mm analog input connector.

- Two red/blue LED pairs are provided on the remote microphone hardware for a total of four LEDs that can be used for user feedback.
- The CFX and ARM Cortex-M3 processors share some memory where attributes can be stored by either. These locations are listed in Table 1. Some parameters also have an interrupt dedicated to them to inform the other processor that a change to the value has occurred while others are simply read each time.

Table 1. Shared Memory Locations

Memory Location	Direction	Interrupt
Volume Level	N/A	N/A
nRF51822 Reset State	ARM Cortex-M3 to CFX	ARM Cortex-M3 to CFX
Radio State	ARM Cortex-M3 to CFX	ARM Cortex-M3 to CFX
Audio Source	ARM Cortex-M3 to CFX	None

- The Ezairo 7100 DSP provides all the supplies for nRF51822, and has access to its reset line which allows Ezairo 7100 to be in full control of the nRF51822 radio.
- The I^2C interface is used as a bidirectional command interface between Ezairo 7100 and nRF51822. The nRF51822 is the I^2C master. Ezairo 7100 uses the I^2C interrupt lines to alert nRF51822 that a message is pending for it to read.
- The SPI interface is used to send the encoded audio packets received over the air by nRF51822 to Ezairo 7100.
- The Wakeup/PLL GPIO is an input to Ezairo 7100 and is used for the following:
  - On initial power-up, nRF51822 sets this line high, and releases it when it has finished initialization. This informs Ezairo 7100 that it can start sending configuration parameters to nRF51822 via the I^2C interface.
  - When wireless audio streaming is enabled, this pin is toggled by nRF51822 every time it sends a new encoded audio packet to Ezairo 7100, which triggers the decoding process and provides a synchronization signal for the PLL.

Figure 2 is a description of the state machine for the system.

Remote Mic User Flow

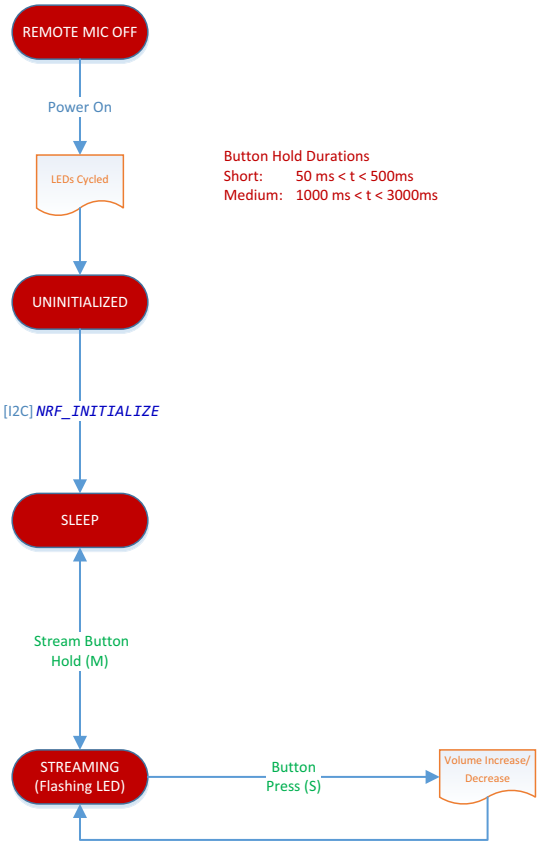


Figure 2. System State Machine

CHAPTER 3

CFX Processor

The CFX processor is responsible for the following functions:

- Turning on the nRF51822 radio IC
- Switching between audio sources
- Toggling the audio stream
- GPIO monitoring for the PLL synchronization signal

The CFX's operation is entirely interrupt-based. The main wait-loop checks the interrupt flag for each of the functions the CFX performs, and handles the flags as necessary.

3.1 Powering the nRF51822 Radio IC

Since all GPIO configuration is performed by the CFX processor, it is also responsible for powering the nRF51822 on. The `CM3_INT2_ISR` interrupt is used by the ARM Cortex-M3 processor to signal to the CFX to power the radio on. When this interrupt is received by the CFX, it sets the `X_CM3_POWER_FLAG` flag, which is handled by the main loop of the CFX.

The `Turn_On_Radio()` function is used to configure the radio power supply and reset signal. The radio is powered off by default.

3.2 Switching Between Audio Sources

The 3.5 mm audio jack switch triggers a GPIO interrupt on the ARM Cortex-M3 processor when an audio source is connected or disconnected (signal is debounced in software). Whenever this interrupt is received, the requested audio source is loaded into shared memory, and the `CM3_INT1_ISR` interrupt is triggered on the CFX. This, in turn, sets the `X_AUDIO_FLAG` flag.

The CFX's main loop handles this flag on its next iteration, and reconfigures the input audio source as requested.

3.3 Toggling Audio Stream

The Ezairo 7150 SL hearing aid firmware uses two push-buttons for user input. One of the buttons is configured to toggle the audio stream while the other toggles BLE. When the stream button is held, the ARM Cortex-M3 processor loads the requested audio stream state into shared memory (`P_RADIO_STATE`), and triggers the `CM3_INT0_ISR` on the CFX. This then sets the `X_STREAM_FLAG` flag.

The CFX's main loop handles this flag, and configures the device to start or stop streaming.

3.4 GPIO Monitoring

The PLL synchronization signal is the only GPIO that is currently monitored by the CFX. The nRF51822 toggles this pin every time that an encoded audio packet is sent to the Ezairo 7100 DSP over the SPI interface. Ezairo 7100 can use this signal to start decoding the next audio packet as well as to synchronize its PLL to nRF51822.

CHAPTER 4

ARM Cortex-M3 Processor

The ARM Cortex-M3 processor's operation is organized into the following main areas:

- System Initialization
- Main System Loop
- I^2C Slave Module
- BLE Module
- SPI Module

The following is a description of the main functions performed by the ARM Cortex-M3 processor on the Ezairo 7100 SoC.

4.1 System Initialization

On system power-up, the CFX processor performs its own initialization, sets nRF51822 to a powered off state, and starts the ARM Cortex-M3 processor.

4.1.1 Hardware Initialization

The ARM Cortex-M3 processor starts by initializing its own peripherals and data structures as necessary. The firmware calls the `Initialize_System()` function to initialize the SPI, I^2C, and GPIOs, before cycling the LEDs to indicate that the system is awake.

Before interrupts are initialized, the button can be used to achieve additional "special" functions that are only available if the button is held while the system is powered on. The firmware takes advantage of this by calling the `Check_Boot_Button()` function during initialization to determine how long (if at all) the button was held on boot-up:

Left Button Held
: Disable BLE security and enter bond mode

Right Button Held
: Delete existing BLE bonding information

Once the button state is read as released, the firmware continues executing. If one of the special modes was requested, the firmware configures itself accordingly and uses LEDs to notify the user.

NOTE: BLE functionality is not used by the remote microphone in Release 1.0 of the Remote Microphone Reference Design firmware.

4.1.2 Radio Initialization

If the radio is being used by default, it can be turned on as part of the system initialization procedure. To power on the radio, its supply (VDBL) must be enabled, and it must be released from reset by using Ezairo 7100 GPIO 10. The nRF51822 radio IC notifies Ezairo 7100 that it has finished hardware initialization by lowering the `RF_WAKEUP_PIN` (GPIO pin 19). Once this signal has been received, the I^2C interface is configured and nRF51822 is ready to read commands from Ezairo 7100.

In the firmware, the `Power_Radio()` function handles turning on the radio, waiting for the radio to complete initialization, and queuing all configuration commands to be sent to nRF51822 in the I^2C queue. For the purpose of

the firmware, these configurations involve initializing the nRF51822 BLE stack and configuring the audio streaming parameters.

4.2 MAIN LOOP

The main loop of the firmware is responsible for monitoring all system flags and handling them as needed. The main loop is also what handles triggering the I^{2}C interrupt line to notify nRF51822 of pending messages.

4.3 BUTTON FLAGS

The `Check_Button_Flags()` function can be used to monitor any button presses from the user. All button presses are debounced, and reported via a flag that can be checked using the `Get_Button_Flag()` function.

The firmware reports and checks the following button presses:

BTN_VOL_UP_FLAG
: Volume up button press

BTN_VOL_DOWN_FLAG
: Volume down button press

BTN_ESB_FLAG
: Volume up hold for <500 ms

NOTE: The buttons can be programmed for multiple purposes if button hold duration is used. The hold duration is captured by the application, but only certain events are handled.

4.4 I^{2}C FLAGS

The main loop must monitor all the I^{2}C interface related flags that might have been set by the I^{2}C library to facilitate the sending/receiving of commands between Ezairo 7100 and nRF51822. The flags that must be monitored are as follows:

- RECEIVE_COMPLETE
  - A command has been received from nRF51822.
  - Parse and handle the command
  - Reset the receive buffer pointer and the state variable.

- SEND_COMPLETE
  - The nRF51822 radio IC has finished reading a command from Ezairo 7100.
  - Reset the I^{2}C interface to clear the next queued byte in the data register of the interface. If this is not done, the next read by nRF51822 starts with the next byte loaded into the data register from the previous transaction.

- SEND_READY
  - The nRF51822 radio IC is requesting data.
  - The first byte in the txBuffer is loaded into the data register, and the ISR in the I^{2}C library handles sending the rest of the data.

- HARD_RESET
  - A fatal error has occurred.

- Reset I^{2}C interface.

NOTE: See the `Check_I2C_Flags` function in the firmware for an implementation of this.

4.5 I^{2}C Queue Management

After checking all the necessary flags, any pending messages in the I^{2}C transmit queue must be processed. When a command must be sent to nRF51822, it is added to the transmit queue by calling the `I2C_Queue_Put()` function. This means that the queue can have multiple messages waiting to be sent to nRF51822 at any given time.

To send a message, check the state of the I^{2}C interface, then:

- If the status is `TX_BUSY`, take no action. This means that the I^{2}C interrupt has been set, and nRF51822 has not yet read the command.
- If the status is `TX_FREE`, nRF51822 has finished reading the last command. Verify that another message is pending using the `I2C_Queue_Status()` function. If a message is pending:
  a. Point the transmit data pointer to it.
  b. Set the I^{2}C interrupt to notify nRF51822 that another message is pending.
  c. Set the status to `TX_BUSY`.

Whenever nRF51822 successfully reads a message, it sends back the `SL_READ_CMPLT` confirmation command indicating that the status can be set to `TX_FREE`, and the interrupt can be cleared.

If the I^{2}C interface is not busy, and no messages are pending, you can put the system into sleep until the next interrupt is received.

4.6 Ezairo 7150 SL 1^{2}C Library

This section describes in detail the workings of the I^{2}C interface implemented between nRF51822 and the ARM Cortex-M3 processor on Ezairo 7100. The I^{2}C interface is responsible for all commands that are sent between the two.

4.6.1 Overview

The nRF51822 acts as the master device of I^{2}C communications, while Ezairo 7100 is the slave. During normal operation, most of the control communication is expected to come from the nRF51822 radio to Ezairo 7100. This is accomplished through I^{2}C writes to Ezairo 7100 with instructions and data.

During initialization and occasionally afterwards, Ezairo 7100 has to initiate communication with the nRF51822 radio. To accomplish this, the Ezairo 7100 can trigger a hardware interrupt on the nRF51822 radio via a shared GPIO (`I2C_INT_PIN` or `GPIO 16`). On its next wake-up, nRF51822 sees the interrupt and performs a standard 2-byte read that contains the pending command header and the byte length of the data pending to be sent. After reading this information, nRF51822 immediately performs a second read that is the size of the data length received in the previous transaction. In this way, Ezairo 7100 can send a variable length command whenever nRF51822 is ready to receive it.

The state machine implemented on Ezairo 7100 to handle these transactions is shown in Figure 3.

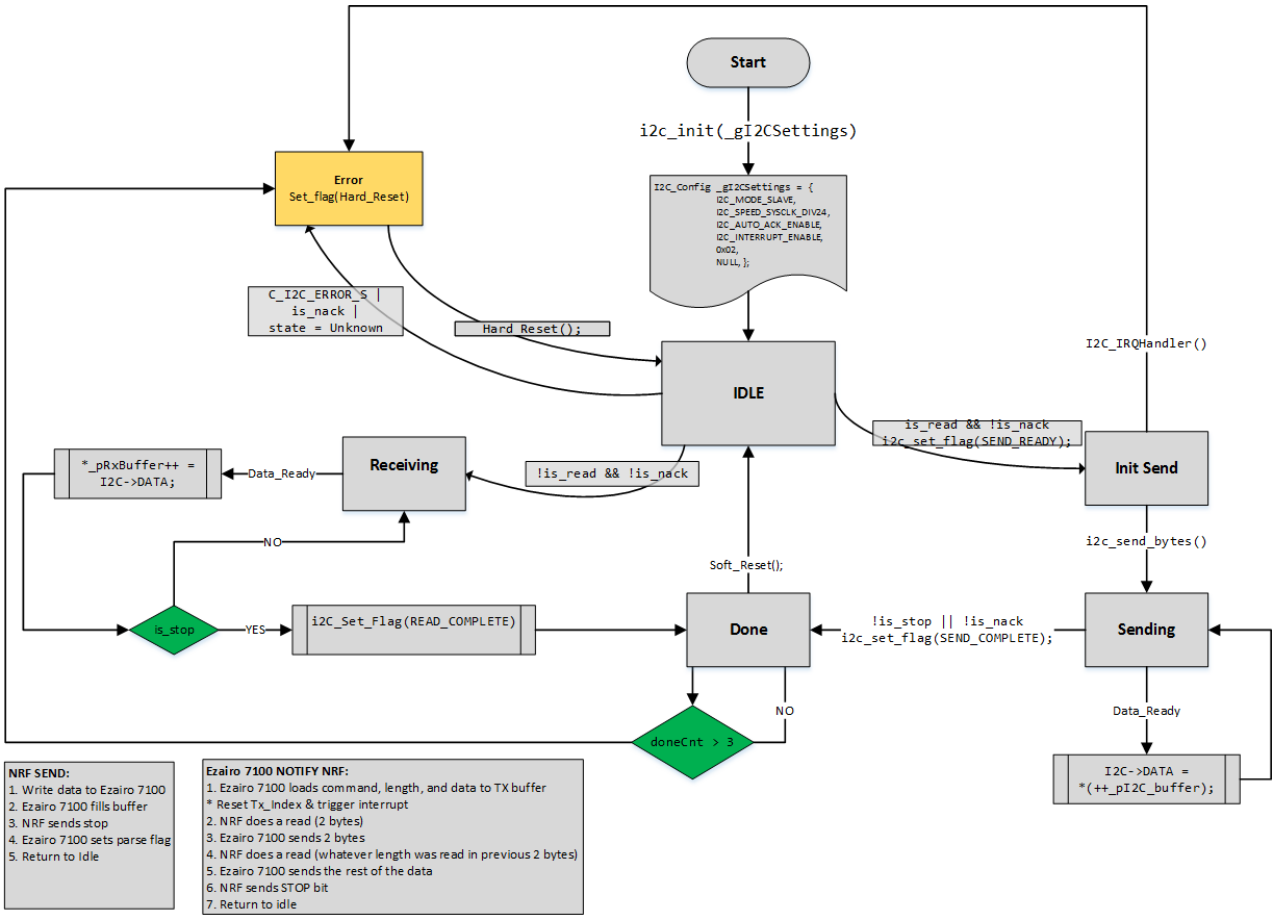


Figure 3. I²C State Flow

4.6.2 Receiving a Command

The I²C driver automatically handles receiving messages from nRF51822 and writing the data into the buffer (RxBuffer[]) available to the rest of the firmware. When a message is successfully read, the I²C driver signals the main application via a flag. To check the new message flag, use the following command:

```
I2C_Get_Flag(RECEIVE_COMPLETE)
```

In the firmware, flag checking and data parsing is implemented using the `Check_I2C_Flags()` and `Parse_Data()` functions.

The incoming I²C commands from nRF51822 take the following format:

Byte 1	Byte 2...Byte n
Command Header	Command Data

4.6.3 Sending a Command

Because Ezairo 7100 is the I²C slave device, it must trigger a read from nRF51822 when a message needs to be sent. This is accomplished by setting the `I2C_INT_PIN` (GPIO 16) high. When nRF5822 detects a rising edge on this DIO, it starts reading from Ezairo 7100 on the next available cycle.

When nRF51822 initiates a read, the I²C driver sets the `SEND_READY` flag indicating that the application must start shifting out data. When this flag is set, call the `I2C_Send_Bytes()` function to start sending data. After the transaction has started, Ezairo 7100 continues incrementing the transmit pointer (_pTxBuffer) and sending the next byte until it receives a NACK from nRF51822.

NOTE: Load the Tx data into the outgoing buffer before the I²C interrupt is triggered.

When nRF51822 has finished reading, it issues a NACK and replies with the `SL_READ_CMPLT` command to indicate that it has successfully read from Ezairo 7100. When the NACK is received, the I²C driver sets the `SEND_COMPLETE` flag, at which point the firmware must reset the I²C interface using the `I2C_Tx_Reset()` function. This is necessary because before the NACK was received, the next byte of data was loaded into the I²C data register and will be sent automatically on the next read from nRF51822 if not cleared. When the `SL_READ_CMPLT` command is received by Ezairo 7100, lower the I²C interrupt line.

The outgoing I²C commands from Ezairo 7100 take the following format:

Byte 1	Byte 2	Byte 3...Byte n
Command Header	Command Data Length	Command Data

4.6.4 I²C Protocol

Table 2 and Table 3 describe all of the available I²C protocol commands. The commands are organized into two groups: TX and RX. Any command with the prefix “NRF_” is a TX command and is meant to be sent to nRF51822 from Ezairo 7100. Any command with the prefix “SL_” is an RX command and can be received by Ezairo 7100 from nRF51822. Any expected reply messages are noted under the format column.

NOTE: nRF51822 replies with the `SL_READ_CMPLT` message to all successfully read messages.

Table 2. Commands from Ezairo 7100 to nRF51822

Header	Name	Description	Format
General access commands for nRF51822 (0x01-0x1F)			
0x01	NRF_CHANGE_CONN_PARAMS	Change advertising and connection parameters	Byte 1: Header (0x01) Byte 2: Data Length (0d12) Byte 3: Advertising Interval MSB Byte 4: Advertising Interval LSB Byte 5: Advertising Timeout MSB Byte 6: Advertising Timeout LSB Byte 7: Min Connection Interval MSB Byte 8: Min Connection Interval LSB Byte 9: Max Connection Interval MSB Byte 10: Max Connection Interval LSB Byte 11: Slave Latency MSB Byte 12: Slave Latency LSB Byte 13: Connection Supervision Timeout MSB Byte 14: Connection Supervision Timeout LSB
0x02	NRF_CHANGE_NAME	Change the radio's advertised name	Byte 1: Header (0x02) Byte 2: Data Length (Radio name length) Byte 3: Radio Name First Character … Byte n: Radio Name Last Character
0x03	NRF_CONFIGURE_STREAM	Change which stream to listen for	Byte 1: Header (0x03) Byte 2: Data Length (3) Byte 3: Stream List Index Byte 4: Stream Address Byte 5: Hearing aid side
0x04	NRF_EZAIRO_VERSION	Send version number of Ezairo 7100 firmware	Byte 1: Header (0x04) Byte 2: Data Length (1) Byte 3: Firmware version number
Control Commands for nRF51822 (0x21-0x3F)			
0x21	NRF_HARDWARE_CFG	Inform nRF51822 of hardware configuration	Byte 1: Header (0x21) Byte 2: Data Length (2) Byte 3: UART Mode Byte 4: Button Configuration Mode
0x22	NRF_INITIALIZE	Instruct nRF51822 to initialize its hardware peripherals and BLE stack	Byte 1: Header (0x21) Byte 2: Data Length (0) Reply: SL_INIT_CMPLT
0x23	NRF_GET_STATE	Request current state of nRF51822 state machine	Byte 1: Header (0x22) Byte 2: Data Length (0) Reply: SL_CURRENT_STATE
0x24	NRF_SLEEP	Put nRF51822 in the sleep state (no BLE or audio streaming)	Byte 1: Header (0x23) Byte 2: Data Length (0)
0x25	NRF_BLE_ON	Turn on BLE on nRF51822	Byte 1: Header (0x24) Byte 2: Data Length (0)

Table 2. Commands from Ezairo 7100 to nRF51822 (Continued)

Header	Name	Description	Format
0x26	NRF_BLE_OFF	Turn off BLE on nRF51822	Byte 1: Header (0x25) Byte 2: Data Length (0)
0x27	NRF_STREAM_ON	Turn on audio stream on nRF51822	Byte 1: Header (0x26) Byte 2: Data Length (0)
0x28	NRF_STREAM_OFF	Turn off audio stream on nRF51822	Byte 1: Header (0x27) Byte 2: Data Length (0)
0x29	NRF_DISCONNECT	Instruct nRF51822 to disconnect from a connected device (central device only)	Byte 1: Header (0x28) Byte 2: Data Length (0)
Commands for custom services/characteristics for nRF51822 (0x41-0x5F)			
0x41	NRF_ADD_SERV	Add a custom service to nRF51822	Byte 1: Header (0x41) Byte 2: Data Length (0d19) Byte 3: Base UUID MSB … Byte 18: Base UUID LSB Byte 19: UUID Offset MSB Byte 20: UUID Offset LSB Byte 21: Service Type
0x42	NRF_ADD_CHAR	Add a custom characteristic to the nRF51822	Byte 1: Header (0x42) Byte 2: Data Length (n) Byte 3: Service Base UUID Offset MSB Byte 4: Service Base UUID Offset LSB Byte 5: Characteristic Data Length Byte 6: Attribute Permission Byte 7: CCCD Permission Byte 8: Properties Byte 9: User Readable Descriptor … Byte 28: User Readable Descriptor Byte 29: Characteristic Initial Value MSB … Byte n: Characteristic Initial Value LSB Reply: SL_HANDLE_INFO
0x43	NRF_UPDATE_CUSTOM	Send updated value to nRF51822 for a custom characteristic	Byte 1: Header (0x023) Byte 2: Data Length (n) Byte 3: Handle MSB Byte 4: Handle LSB Byte 5: Characteristic Value MSB Byte n: Characteristic Value LSB
Commands for predefined services/characteristics for nRF51822 (0x61-0x7F)			
0x61	NRF_ENABLE_BAS	Enable Battery Alert Service	Byte 1: Header (0x61) Byte 2: Data Length (0)
0x62	NRF_ENABLE_RMAS	Enable Remote Microphone Audio Stream Service	Byte 1: Header (0x62) Byte 2: Data Length (0)

Table 2. Commands from Ezairo 7100 to nRF51822 (Continued)

Header	Name	Description	Format
0x63	NRF_ENABLE_IAS	Enable Immediate Alert Service	Byte 1: Header (0x63) Byte 2: Data Length (0)
0x64	NRF_ENABLE_HAS	Enable Hearing Aid Status Service	Byte 1: Header (0x64) Byte 2: Data Length (0)
0x65	NRF_UPDATE_HAS_MODE	Update Hearing Aid Mode characteristic	Byte 1: Header (0x65) Byte 2: Data Length (0d1) Byte 3: New Mode
0x66	NRF_UPDATE_BAS	Update battery level characteristic	Byte 1: Header (0x66) Byte 2: Data Length (0d1) Byte 3: Battery Level

Table 3. Commands from nRF51822 to Ezairo 7100

Header	Name	Description	Format
Control commands for Ezairo7100 (0x81-0x9F)			
0x81	SL_INIT_CMPLT	nRF51822 has finished initializing its hardware peripherals and software stack	Byte 1: Header (0x81)
0x82	SL_READ_CMPLT	nRF51822 has finished reading a command from Ezairo 7100	Byte 1: Header (0x82)
0x83	SL_CONNECTED	nRF51822 has established a BLE connection with a peripheral device	Byte 1: Header (0x83)
0x84	SL_DISCONNECTED	nRF51822 has experienced a BLE disconnection	Byte 1: Header (0x84)
0x85	SL_NEW_BOND	nRF51822 has established a new bond with a peripheral device	Byte 1: Header (0x85)
0x86	SL_NO_WHITELIST	nRF51822 has no whitelist data and is unable to advertise/scan	Byte 1: Header (0x86)
0x87	SL_TIMEOUT	nRF51822 advertising/scanning has timed out	Byte 1: Header (0x87)
0x88	SL_ESB_DATA_SENT	nRF51822 has sent the audio stream parameters to the hearing aid device (remote microphone only	Byte 1: Header (0x88)
0x89	SL_CURRENT_STATE	nRF51822 is sending current state of state machine. This occurs on every state transition.	Byte 1: Header (0x89) Byte 2: Current State
Commands for custom services/characteristics for Ezairo 7100 (0xA1-0xBF)			
0xA1	SL_HANDLE_INFO	Handle value assigned to a custom characteristic	Byte 1: Header (0xA1) Byte 2: Handle MSB Byte 3: Handle LSB Byte 4: Characteristic UUID MSB … Byte 19: Characteristic UUID LSB

Table 3. Commands from nRF51822 to Ezairo 7100 (Continued)

Header	Name	Description	Format
0xA2	SL_UPDATE_CUSTOM	Update the value of a custom characteristic	Byte 1: Header (0xA2) Byte 2: Handle MSB Byte 3: Handle LSB Byte 4: Updated Value MSB … Byte n: Updated Value LSB
0xA3	SL_UPDATE_HAS_MODE	Update the mode characteristic of the hearing aid status service	Byte 1: Header (0xA3) Byte 2: Updated Mode
0xA4	SL_UPDATE_HAS_STATUS	Update the audio stream status characteristic of the hearing aid status service	Byte 1: Header (0xA4) Byte 2: Updated Mode
Commands for predefined services/characteristics for Ezairo 7100 (0xC1-0xDF)			
0xC1	SL_UPDATE_IAS	Update the value of the immediate alert service	Byte 1: Header (0x61) Byte 2: Alert Value
Reserved Commands (0xF0-0xFF)			
0xF1	ERROR_REPORT	Error has occurred on nRF51822	Byte 1: Header (0x61) Byte 2: Alert Value

4.6.5 I²C Library Functions

The available functions in the I²C library are listed in Table 4. Each function is explained in detail in the following subsections.

Table 4. Function List

Function	Description
I2C_Initialize	Initialize the I²C interface
I2C_IRQHandler	Handler for I2C interrupts.
I2C_Clear _Interrupt	Clears the I²C interrupt, and clears the I²C busy state to allow a new interrupt to occur
I2C_Set_Interrupt	Triggers the I²C interrupt to indicate to nRF51822 that a message is pending
I2C_Send_Bytes	Start shifting out the transmit buffer data to nRF51822
I2C_Queue_Put	Adds an I²C message to the TxQueue
I2C_Queue_Get	Returns a pointer to the start of the next TX message
I2C_Queue_Status	Checks if the Tx queue is empty
I2C_Queue_Clear	Clears the Tx queue
I2C_Set_Flag	Sets the indicated I²C flag
I2C_Get_Flag	Returns the status of the requested I²C flag (clears flag)
I2C_Hard_Reset	Reset the I²C interface and this library's internal state
I2C_Tx_Reset	Reset Tx related I²C variables and state
I2C_Rx_Reset	Reset Rx related I²C variables and state

4.6.5.1 I²C Initialize

Type	Function
Include File	#include <i2c.h>
Source File	i2c.c
Template	`void I2C_Initialize( I2C_Config config )`
Description	Initialize the I²C interface

Inputs	`I2C_Config _gI2CSettings = {`
	`  I2C_MODE_SLAVE,`
	`  I2C_SPEED_SYSCLK_DIV6,`
	`  I2C_AUTO_ACK_ENABLE,`
	`  I2C_INTERRUPT_ENABLE,`
	`  I2C_SLAVE_ADDRESS,`
	`  NULL}`
	`;`

Outputs	None
Assumptions	None
Example	`/* Initialize I2C interface */` `I2C_Initialize(_gI2CSettings);`

4.6.5.2 I²C IRQHandler

Type	Function
Include File	#include <i2c.h>
Source File	i2c.c
Template	`void I2C_IRQHandler( void )`
Description	Handler for I²C interrupts
Inputs	None
Outputs	None
Assumptions	None
Example	N/A

4.6.5.3 I²C Clear Interrupt

Type	Function
Include File	#include <i2c.h>
Source File	i2c.c
Template	`void I2C_Clear_Interrupt(void)`
Description	Clears the I²C interrupt, and clears the I²C busy state to allow for a new interrupt to occur
Inputs	None
Outputs	None
Assumptions	None

Example	`/* nRF51822 is done doing the I2C read from NP5 */`
	`case SL_READ_CMPLT:`
	`  I2C_Clear_Interrupt();`
	`  break;`

4.6.5.4 I²C Set Interrupt

Type	Function
Include File	#include <i2c.h>
Source File	i2c.c
Template	`void I2C_Set_Interrupt(void)`
Description	Triggers the I²C interrupt to indicate to nRF51822 that a message is pending
Inputs	None
Outputs	None
Assumptions	None
Example	`/* If the TxQueue has something in it then point the Tx pointer to * it and trigger the nrf interrupt */ if (I2C_Queue_Status() != I2C_QUEUE_EMPTY) { _pTxBuffer = I2C_Queue_Get(); /* Queue next message */ I2C_Set_Interrupt(); }`

4.6.5.5 I²C Send Bytes

Type	Function
Include File	#include <i2c.h>
Source File	i2c.c
Template	`i2c_error_t I2C_Send_Bytes(void)`
Description	Start shifting out the transmit buffer data to nRF51822
Inputs	None
Outputs	i2c_error_t - error message
Assumptions	None
Example	`/* nRF51822 has started a read in response to us raising the interrupt line. Start sending out the data in the txBuffer */ if (I2C_Get_Flag(SEND_READY)) { I2C_Send_Bytes(); }`

4.6.5.6 I²C Queue Put

Type	Function
Include File	#include <i2c.h>

Source File	i2c.c
Template	`uint8_t I2C_Queue_Put(uint8_t * dataStart, uint8_t dataLen)`
Description	Adds an I²C message to the TxQueue
Inputs	`uint8_t * dataStart` - Pointer to array containing data to add `uint8_t dataLen` - Number of bytes to add to queue
Outputs	`QUEUE_FULL` `ERROR_NONE`
Assumptions	None

Example	`txBuffer[0] = (uint8_t)NRF_INITIALIZE;`
	`txBuffer[1] = 0x00; /* Data Length = 0 */`
	`I2C_Queue_Put(&txBuffer[0], 2);`

4.6.5.7 I²C Queue Get

Type	Function
Include File	#include <i2c.h>
Source File	i2c.c
Template	`volatile uint8_t * I2C_Queue_Get( void )`
Description	Returns a pointer to the start of next TX message
Inputs	None
Outputs	`volatile uint8_t *` - Pointer to the next message to be sent
Assumptions	Queue is not empty (use `Queue_Status()` function)
Example	`_pTxBuffer = I2C_Queue_Get(); /* Queue next message */`

4.6.5.8 I²C Queue Status

Type	Function
Include File	#include <i2c.h>
Source File	i2c.c
Template	`i2c_error_t I2C_Queue_Status( void )`
Description	Checks if transmit queue is empty
Inputs	None
Outputs	i2c_error_t - error message
Assumptions	None

Example	`/* If the TxQueue has something in it then point the Tx pointer to`
	`* it and trigger the nRF interrupt */`
	`if (I2C_Queue_Status() != QUEUE_EMPTY){`
	`_pTxBuffer = I2C_Queue_Get(); /* Queue next message */`
	`Set_Interrupt();`
	`}`

4.6.5.9 I²C Queue Clear

Type	Function
Include File	#include <i2c.h>
Source File	i2c.c
Template	`void I2C_Queue_Clear( void )`
Description	Clears the transmit queue
Inputs	None
Outputs	None
Assumptions	None
Example	`/* Entering new mode - clear all previous messages from Tx queue */` `I2C_Queue_Clear();`

4.6.5.10 I²C Set Flag

Type	Function
Include File	#include <i2c.h>
Source File	i2c.c
Template	`void I2C_Set_Flag(uint32_t flag)`
Description	Sets the indicated I²C flag
Inputs	`uint32_t flag - flag to set`
Outputs	None
Assumptions	None

Example	`/* Last byte received */`
	`if (is_stop){`
	`  _gI2CState = I2C_STATE_DONE;`
	`  I2C_Set_Flag(RECEIVE_COMPLETE);`
	`}`

4.6.5.11 I²C Get Flag

Type	Function
Include File	#include <i2c.h>
Source File	i2c.c
Template	`uint8_t I2C_Get_Flag(uint32_t flag)`
Description	Returns the status of the requested I²C flag (clears flag)
Inputs	`uint32_t flag - flag to get`
Outputs	uint8_t - Value of flag
Assumptions	None

Example	`if (I2C_Get_Flag(RECEIVE_COMPLETE)) {`
	`  Parse_Data();`
	`  I2C_Rx_Reset();`
	`}`

4.6.5.12 I²C Hard Reset

Type	Function
Include File	#include <i2c.h>
Source File	i2c.c
Template	`void I2C_Hard_Reset( void )`
Description	Reset the I²C interface and the library's internal state
Inputs	None
Outputs	None
Assumptions	None

Example	`/* I2C Error has occurred - Perform reset of I2C interface */`
	`if (I2C_Get_Flag(HARD_RESET)) {`
	`  I2C_Hard_Reset();`
	`}`

4.6.5.13 I²C Tx Reset

Type	Function
Include File	#include <i2c.h>
Source File	i2c.c
Template	`void I2C_Tx_Reset( void )`
Description	Reset transmit-related I²C variables and state
Inputs	None
Outputs	None
Assumptions	None
Example	`/* Perform reset after nRF51822 has finished reading */` `I2C_Tx_Reset();`

4.6.5.14 I²C Rx Reset

Type	Function
Include File	#include <i2c.h>
Source File	i2c.c
Template	`void I2C_Rx_Reset( void )`
Description	Reset receive-related I²C variables and state
Inputs	None
Outputs	None
Assumptions	None

Example	`if (I2C_Get_Flag(RECEIVE_COMPLETE)){`
	`  Parse_Data();`
	`  I2C_Rx_Reset();`
	`}`

4.6.6 Ezairo 7150 SL Library

This section describes the working BLE library included with the firmware. This library contains BLE and audio stream related functions and is implemented on the ARM Cortex-M3 processor within the Ezairo 7150 SL hybrid module.

4.6.6.1 Overview

The BLE library uses the I^2C library to provide convenient functions to perform basic radio configuration and control. The library allows you to initialize the radio, turn BLE and audio streaming on and off, and put nRF51822 into a sleep state. The hearing aid firmware application uses the BLE library throughout for initialization of the radio and for control of the radio state flow.

4.7 BLE Library Functions

The available functions in the BLE Support Library are listed in Table 5. Each function is explained in detail in the following subsections.

Table 5. List of BLE Functions

Function	Description
Configure_Audio_Stream	Adds a message to the queue to tell the nRF51822 to configure the audio stream parameters based on the defines
Change_Radio_Name	Adds a message to the queue to tell the nRF51822 to change what target device name it is looking for over BLE
Initialize_Radio	Adds a message to the queue to tell nRF51822 to initialize itself (peripherals, BLE stack, BLE services, connection parameters)
BLE_On	Adds a message to the Tx queue to tell the nRF51822 to start advertising
BLE_Off	Adds a message to the Tx queue to tell the nRF51822 to turn off BLE functionality
Stream_On	Start listening for the audio stream
Stream_Off	Stop listening for the audio stream
NRF51822_Sleep	Adds a message to the Tx queue to tell nRF1822 to go to sleep. This turns off BLE and ESB.

4.7.1 Configure Audio Stream

Type	Function
Include File	#include <ble.h>
Source File	ble.c
Template	`void Configure_Audio_Stream( void )`
Description	Adds a message to the queue to tell the nRF51822 to configure the audio stream parameters based on the defines
Inputs	None
Outputs	None
Assumptions	None
Example	`/* Configure audio stream channel list and address */` `Configure_Audio_Stream();`

4.7.2 Change Radio Name

Type	Function
Include File	#include <ble.h>
Source File	ble.c
Template	`void Change_Radio_Name(uint8_t * _pNewRadioName)`
Description	Adds a message to the queue to tell the nRF51822 to change what target device name it is looking for over BLE
Inputs	`uint8_t * _pNewRadioName` - pointer to radio name character array
Outputs	None
Assumptions	None
Example	`/* Configure radio name */` `Change_Radio_Name(radioName);`

4.7.3 Initialize Radio

Type	Function
Include File	#include <ble.h>
Source File	ble.c
Template	`void Initialize_Radio(bond_mode_t bondMode);`
Description	Adds a message to the queue to tell nRF51822 to initialize itself (peripherals, BLE stack, BLE services, connection parameters)
Inputs	`bond_mode_t` - Bond deletion enabled/disabled
Outputs	None
Assumptions	None
Example	`/* Add a message to the queue to initialize the nRF51822 */` `Initialize_Radio(SAVE_BOND_INFO);`

4.7.4 BLE On

Type	Function
Include File	#include <ble.h>
Source File	ble.c
Template	`void BLE_On(sec_mode_t secMode);`
Description	Adds a message to the Tx queue to tell nRF51822 to start advertising
Inputs	`sec_mode_t` - Whitelist or open advertising
Outputs	None
Assumptions	None
Example	`/* Start advertising with whitelist */` `BLE_On(BLE_WHITELIST_ENABLE);`

4.7.5 BLE Off

Type	Function
Include File	#include <ble.h>
Source File	ble.c
Template	`void BLE_Off(void);`
Description	Adds a message to the Tx queue to tell nRF51822 to turn off BLE functionality
Inputs	None
Outputs	None
Assumptions	None
Example	`/* Add a message to the queue to tell the nRF51822 to turn off BLE */` `BLE_Off();`

4.7.6 Stream On

Type	Function
Include File	#include <ble.h>
Source File	ble.c
Template	`void Stream_On(void);`
Description	Start listening for the audio stream
Inputs	`sec_mode_t` - Whitelist or open advertising
Outputs	None
Assumptions	None

Example	`/* Start the audio stream and capture the data */`
	`Sys_DMA_Channel1_Enable();`
	`Stream_On();`

4.7.7 Stream Off

Type	Function
Include File	#include <ble.h>
Source File	ble.c
Template	`void Stream_Off(void);`
Description	Stop listening for the audio stream
Inputs	None
Outputs	None
Assumptions	None

Example	`/* Disable the audio stream */`
	`Sys_DMA_Channel1_Disable();`
	`Stream_Off();`

4.7.8 nRF51822 Sleep

Type	Function
Include File	#include <ble.h>
Source File	ble.c
Template	`void NRF51822_Sleep(void);`
Description	Adds a message to the Tx queue to tell nRF51822 to go to sleep. This turns off BLE and ESB.
Inputs	None
Outputs	None
Assumptions	None
Example	`/* NexState = Sleep */` `NRF51822_Sleep();`

CHAPTER 5

NRF51822 Radio IC

This chapter provides a high-level overview of the nRF51822's operation, and provides details on how the Ezairo 7100 DSP interacts with it.

The nRF51822 radio IC has a state machine that handles I^2C transactions, power management, and state changes. When nRF51822 is booted, it configures its GPIOs and sets the wakeup line high to indicate to Ezairo 7100 that it is not yet ready. Then, nRF51822 initializes its hardware peripherals. Once the system has been fully configured, it lowers the wake-up line, indicating to Ezairo 7100 that it has completed booting. At this point, nRF51822 is in its *Uninitialized* state, meaning that only the hardware necessary to communicate with Ezairo 7100 has been initialized. It waits for an interrupt to occur, then Ezairo 7100 can start to send any I^2C commands to further configure nRF51822.

After it has been configured, nRF51822 is ready to start receiving control commands. Ezairo 7100 can instruct nRF51822 to transition between its states by sending the correct commands to enable/disable the audio stream. If an invalid transition is requested, no state transition occurs and nRF51822 responds with the current state.

Ezairo 7150 SL Remote Microphone Reference Design Firmware Manual

ON Semiconductor and ON are registered trademarks of Semiconductor Components Industries, LLC (SCILLC). Ezairo is a registered trademark of SCILLC. Cortex and ARM are registered trademarks of ARM Limited. Bluetooth is a registered trademark of Bluetooth SIG, Inc. SCILLC owns the rights to a number of patents, trademarks, copyrights, trade secrets, and other intellectual property. A listing of SCILLC's product/patent coverage may be accessed at www.onsemi.com/site/pdf/Patent-Marking.pdf. SCILLC reserves the right to make changes without further notice to any products herein. SCILLC makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does SCILLC assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation special, consequential or incidental damages. "Typical" parameters which may be provided in SCILLC data sheets and/or specifications can and do vary in different applications and actual performance may vary over time. All operating parameters, including "Typicals" must be validated for each customer application by customer's technical experts. SCILLC does not convey any license under its patent rights nor the rights of others. SCILLC products are not designed, intended, or authorized for use as components in systems intended for surgical implant into the body, or other applications intended to support or sustain life, or for any other application in which the failure of the SCILLC product could create a situation where personal injury or death may occur. Should Buyer purchase or use SCILLC products for any such unintended or unauthorized application, Buyer shall indemnify and hold SCILLC and its officers, employees, subsidiaries, affiliates, and distributors harmless against all claims, costs, damages, and expenses, and reasonable attorney fees arising out of, directly or indirectly, any claim of personal injury or death associated with such unintended or unauthorized use, even if such claim alleges that SCILLC was negligent regarding the design or manufacture of the part. SCILLC is an Equal Opportunity/ Affirmative Action Employer. This literature is subject to all applicable copyright laws and is not for resale in any manner.

PUBLICATION ORDERING INFORMATION

LITERATURE FULFILLMENT:
Literature Distribution Center for ON Semiconductor
P.O. Box 5163, Denver, Colorado 80217 USA
Phone: 303-675-2175 or 800-344-3860 Toll Free USA/Canada
Fax: 303-675-2176 or 800-344-3867 Toll Free USA/Canada
Email: orderlit@onsemi.com

N. American Technical Support: 800-282-9855 Toll Free USA/Canada
Europe, Middle East and Africa Technical Support: Phone: 421 33 790 2910
Japan Customer Focus Center: Phone: 81-3-5817-1050

ON Semiconductor Website: www.onsemi.com

Order Literature: http://www.onsemi.com/orderlit

For additional information, please contact your local Sales Representative

M-20825-001